

FRED Guide Buffer: Sommaire

[Présentation](#)

[Le registre de condition](#)

[Le T Commande](#)

[Le saut Commandes](#)

[Le U \(Jusqu'à\) Commandes](#)

[Nombre Registres](#)

[Directives flux](#)

[Commentaires](#)

[Lire Listes d'](#)

[exécution FRED Tampons](#)

[options](#)

[Exemples de programmes tampon](#)

[Exemple 1: Réduction d' un fichier de carte](#)

[Exemple 2: Un fichier Mémo](#)

[Exemple 3: A Write Buffer](#)

[Exemple 4: Init tampons](#)

[Exemple 5: Un tampon pour générer tampons](#)

[Exemple 6: Arbres racinées](#)

introduction

Ce manuel est destiné à expliquer les bases de l' écriture des tampons FRED. Nous supposons que vous avez déjà appris les bases de l' utilisation FRED comme indiqué dans le [Guide Tutorial FRED](#) . Il sera également utile si vous pouvez vous référer à d' [autres documents FRED](#) . Autres pages FRED donnent des explications plus détaillées de la plupart des choses que nous discutons ici.

Ce manuel est divisé en deux parties. La première partie présente un certain nombre de commandes FRED qui ne sont pas décrites dans le guide de tutoriel. Ces commandes sont rarement utilisées dans interactive d'édition de texte normal, mais ils sont importants lors de l'écriture des programmes tampons. La deuxième partie de ce guide présente un certain nombre de programmes de tampons FRED qui démontrent comment écrire et utiliser des tampons FRED.

Un tampon FRED (ou programme de tampon) est vraiment une chose très simple. Quiconque a déjà utilisé FRED du tout sait qu'un tampon est tout simplement un endroit pour stocker le texte. Le texte que vous stockez dans la mémoire tampon peut être la source d'un programme Fortran, le contenu du rapport annuel d'une entreprise, ou toute autre chose que vous choisirez. Lorsque vous écrivez un programme de mémoire tampon, le texte qui est stocké dans la mémoire tampon se compose d'un certain nombre de FRED commandes; après tout, les commandes FRED sont vraiment seulement des chaînes de caractères comme tout autre texte.

Une fois que vous avez les commandes stockées dans la mémoire tampon, vous pouvez avoir FRED exécuter les commandes, comme si vous les avez tapés directement depuis le terminal. Bien sûr, tout l' avantage est que vous *ne* devez les taper directement à partir du terminal. Non seulement ceci évite beaucoup de frappe répétitive, mais il évite le risque constant de faire une erreur de frappe qui encrasser les choses. Lorsque vous stockez FRED commandes dans un tampon, vous pouvez modifier ces commandes comme tout autre texte. De cette façon, vous ne devez jamais exécuter les commandes jusqu'à ce que vous êtes sûr qu'ils ont été correctement saisi.

Il est important de reconnaître ce tampon programmes peuvent et ne peuvent pas faire. FRED est un excellent langage pour manipuler le texte. Il a également limité les capacités arithmétiques et des structures de contrôle (par exemple, des sauts et "jusqu'à ce que" les boucles). Cependant, FRED a pas de remplacement pour les langages de programmation tels que Fortran, APL, ou Pascal; il n'a pas été conçu pour le "nombre-croquant" ou de prendre des décisions logiques complexes. Pourtant, FRED peut être extrêmement utile dans une grande variété d'applications, comme on le verra dans le reste de ce manuel.

Le registre de condition

Le registre de condition est une caractéristique interne de FRED qui peut être réglé sur TRUE ou FALSE. Un certain nombre de commandes de définir la valeur de ce registre. Par exemple, la commande S va régler le registre de condition à TRUE si elle fait avec succès une substitution; si elle ne parvient pas à faire un changement, il établira le registre de condition à FALSE. Le R et les commandes W régler le registre de condition à TRUE si la lecture ou l'écriture a réussi; s'il y avait une sorte d'accès ou I / O erreur, le registre de condition est définie sur FALSE.

Un certain nombre de commandes permet de tester le registre de condition pour voir si elle est VRAI ou FAUX. Par exemple, la commande JT saute à une nouvelle ligne si le registre de condition est TRUE. De cette façon, vous pouvez contrôler l'ordre dans lequel votre programme exécute des instructions.

Le commandement T

L'un des procédés les plus courants dans FRED examine une ligne de texte pour voir si elle contient une certaine chaîne de caractères. Lorsque vous utilisez FRED interactive, vous pouvez juste regarder la ligne; lorsque vous écrivez un tampon FRED, vous pouvez utiliser la commande de test. La commande d'essai a la forme

`t / <motif> /`

où <motif> est tout motif FRED valide. Si la ligne actuelle contient une chaîne qui correspond à <motif>, le registre d'état est réglé sur TRUE; sinon, il est réglé sur FALSE.

Vous pouvez également spécifier une plage d'adresses pour la commande T comme dans

`1, $ t / $ /`

Les tests de commande ci-dessus pour voir si l'une des lignes dans le tampon courant se terminent par un caractère blanc. Le pointeur de la ligne en cours "." sera mis à la première ligne dans la plage où le motif se trouve.

Une variation sur la commande d'essai a la forme

`T ~ / <motif> /`

Cela va régler le registre de condition à TRUE si le <motif> ne se trouve pas dans la ligne courante; sinon, le registre de condition est définie sur FALSE. Cette forme de la commande de test peut également prendre une plage d'adresses comme dans

`-1 $, $ T ~ / xxx /`

La commande "T ~" établira le pointeur de la ligne en cours "." à la première ligne dans la gamme qui ne contient pas une chaîne correspondant au motif donné.

Normalement, la commande recherche d'essai pour son modèle à partir de la première adresse dans sa gamme et se terminant à la seconde. Vous pouvez avoir cette recherche revenir en arrière en inversant l'ordre des adresses. Par exemple,

`$, 1t /: /`

commence à la dernière ligne du tampon et recherche vers l'arrière pour un ":". Ainsi, il mettra le pointeur de la ligne en cours "." à la dernière ligne dans la mémoire tampon qui contient un «:».

Comme vous le verrez dans les sections à venir, les commandes de test sont très souvent utilisés dans les programmes de tampons.

Les commandes Jump

Les commandes FRED Jump sont similaires à GOTO dans d'autres langages de programmation. Les différents types de sauts sont décrits dans les sections suivantes.

Le JM (Aller message) Commande:

La commande Jump du message traite tout simplement ce que suit comme un message qui doit être imprimé à la borne. Par exemple,

```
jm / Bonjour il /
```

gravures "Bonjour à tous" sur le terminal. A la place des caractères "/", vous pouvez utiliser tous les caractères non-alphanumériques pour délimiter le message.

La commande JM met un caractère de nouvelle ligne à la fin de son message. La commande JP (Jump Prompt) fait exactement la même chose que JM sauf qu'il omet le caractère de nouvelle ligne. Par exemple,

```
jp; Voulez-vous un manuel (oui ou non) ?;
```

imprimera

Voulez-vous un manuel (oui ou non)?

et ensuite attendre une entrée sans passer à une nouvelle ligne. Ceci est pratique lorsque vous invitant l'utilisateur pour l'entrée.

Ligne Sauts:

Un des plus simples des commandes Jump est le "saut de ligne". Ces commandes indiquent FRED pour ignorer toutes les autres commandes qui suivent sur la même ligne, en fonction de la valeur du registre de condition. JT (Aller si True) dit FRED d'ignorer le reste de la ligne si le registre de condition est TRUE; JF (Aller si False) dit FRED d'ignorer le reste de la ligne si le registre de condition est FALSE. Par exemple, considérons la ligne suivante:

```
r / file1 jt r / file2
```

Les premières tentatives de commande R pour lire un fichier permanent nommé "/ file1". Si la lecture réussit, le registre de condition sera réglé sur TRUE; parce que le registre de condition est TRUE, le JT devra FRED ignorer la deuxième lecture. Cependant, si la première lecture échoue, le registre de condition sera réglé à FALSE; FRED ne saute pas au JT et tentera donc de la deuxième lecture. Ainsi, la déclaration ci-dessus va lire un fichier permanent "fichier1" le cas échéant; sinon, il va chercher "file2".

Il faut signaler que la commande de lecture ci-dessus avec le JT ne serait utile dans un tampon. Si une commande échoue Lire en mode interactif, FRED émet une erreur et arrête immédiatement sans exécuter le reste de la ligne. Si un échec de lecture lors de l'exécution d'un tampon, vous ne recevrez pas le message d'erreur habituelle; FRED sera simplement régler le registre de condition à FALSE et continuer l'exécution des commandes.

Ci-dessous, nous donnons un exemple de la commande JF.

```
* T /% / jf jm? Ce tampon a un caractère "%".?
```

Les tests de commande d'essai ci-dessus chaque ligne dans le tampon pour un caractère "%". Si elle ne trouve pas, il établit le registre de condition à FALSE et JF saute la commande JM; s'il y a un "%" dans le tampon, le registre d'état est réglé sur TRUE, la JF ne saute pas, et JM imprime le message donné.

Sauts Étiqueté:

Une étiquette FRED a la forme

@ (<Nom>)

où <nom> respecte les mêmes restrictions que les noms de tampons. Comme les noms de tampons, les parenthèses ne sont pas nécessaires dans les noms de marque si le nom est un seul caractère long et l'OI (option est en vigueur.

La commande

J (<nom>)

saute à l'étiquette "@ (<nom>)". L'étiquette doit être dans le même tampon que la commande J, mais il peut venir avant ou après la commande dans la mémoire tampon. La recherche d'une étiquette commence à la ligne immédiatement après l'instruction J, va au fond de la mémoire tampon, et enroule autour de la partie supérieure si nécessaire. Ainsi, une recherche d'étiquette fonctionne comme une recherche de modèle ordinaire. La commande

J (<nom>) T

va sauter à "@ (<nom>)" si le registre de condition est TRUE et

J (<nom>) F

sautera si le registre de condition est FALSE. Par exemple, considérez la séquence de commandes suivante.

```
r / file1 j (go) t
r / file2 j (go) t
jm: Vous ne pouvez pas lire les deux / file1 ou / file2 .:
q
@(aller)
<plusieurs commandes ici>
```

La première ligne essaie de lire "/ file1"; si cela réussit, le registre d'état est réglé sur TRUE et FRED va sauter vers le bas pour "@ (aller)". Sinon, FRED va à la deuxième ligne et essaie de lire "/ file2"; si cette lecture réussit, le registre de condition sera réglé sur TRUE et FRED aussi sauter à "@ (aller)". Si ni "/ file1" ni "/ file2" peut être lu, le JM imprime son message et la commande Q vont dire FRED cesser de fumer. Ce type de séquence est souvent trouvée dans des tampons FRED.

Le JE (Jump Exit) Commande:

La commande Jump de sortie est une combinaison du Jump message et que la commande Q, dans une certaine mesure. Comme la commande JM, il a la capacité d'imprimer un message; Cependant, une fois que ce message a été imprimé, la commande JE termine l'exécution de tous les tampons. Habituellement, cela signifie que FRED va revenir en mode interactif et prendra ses commandes directement à partir du terminal; Cependant, si l'option O + Q est en vigueur, la commande JE quittera FRED entièrement et revenir au niveau du système. Par exemple,

```
r / myfile jt je / Vous ne pouvez pas lire file./
```

va tenter de lire "/ myfile". Si la lecture est réussie, le registre de condition est réglé sur TRUE et le JT va sauter à la ligne suivante. Cependant, si la lecture échoue, le JE sera exécutée. Le message sera imprimé et FRED reviendra à obtenir ses commandes à partir du terminal plutôt que le programme de la mémoire tampon. Si vous voulez ce genre d'erreur pour aboutir à quitter tout à fait FRED, vous pourriez dire

```
o + q
r / myfile jt je / Vous ne pouvez pas lire file./
```

Il existe plusieurs autres types de commandes de saut disponibles dans FRED. Voir le manuel de référence pour plus de détails FRED.

Le U (jusqu'à ce que) Commandes

Les commandes sont des commandes Jusqu'à répétition. Ils peuvent être utilisés pour exécuter des boucles et des boucles conditionnelles.

La forme la plus simple de la commande Jusqu'à est

```
U <nombre> <liste de commandes>
```

Cette exécute la <liste des commandes> un total de <nombre> fois. Par exemple,

```
u2 s-1 / "/" '/'
```

va changer les deux dernières guillemets sur une ligne entre guillemets simples. Cela changerait

```
"Bonjour," dit-il, «comment vas-tu?"
```

dans

```
"Bonjour," dit-il, comment vas-tu?
```

Un nombre quelconque de commandes valides FRED peut apparaître dans la <liste des commandes>. Notez cependant que la liste ne se prolonge jusqu'au premier caractère unescaped nouvelle ligne. Si vous voulez des caractères de nouvelle ligne dans <liste de commandes>, vous devrez leur échapper en les faisant précéder "\C", comme dans

```
u2 jm; Elle vous aime; \ c
    jm; Ouais, ouais, ouais;
```

La boucle de U2 comprend à la fois JM commandes et donc permet d'imprimer

```
Elle t'aime
Ouais ouais ouais
Elle t'aime
Ouais ouais ouais
```

Notez que vous auriez pu faire la même chose avec la commande

```
u2 jm / Elle vous aime / jm / Ouais, ouais, ouais /
```

Une autre commande Jusqu'à est UT (Jusqu'à True). Cela a la forme

```
UT <liste de commandes>
```

Cela va régler le registre de condition à FALSE et puis répétez la <liste de commandes> jusqu'à ce que le registre de condition se trouve être TRUE à la fin de l'une des répétitions. De même, l'UF (Jusqu'à False) commande définir le registre de condition à TRUE et répéter une liste de commandes jusqu'à ce que le registre de condition est FALSE à la fin de l'une des répétitions. Par exemple,

```
uf s1 /.@// t /.@/
```

fonctionne sur la ligne actuelle et supprime tous les cas où le caractère "@" suit un caractère @ non. Ce type d'enseignement est pratique dans les cas où vous avez été par erreur en utilisant le "@" comme un caractère caractère de suppression. La commande S dans la boucle UF supprime la première instance de caractère suivi d'un "@". La commande T teste ensuite pour voir s'il y a des personnages plus "@" dans la ligne. Si oui, le registre d'état est réglé sur TRUE et la boucle est exécutée à nouveau; s'il n'y a plus - paires (non @, @), le registre de condition est définie sur FALSE, et l'UF ne sera pas répéter la boucle. Si ce qui précède UF ont été appliquées à la ligne

```
Hellab @@ o thr @ avant.
```

ce serait une boucle par trois fois, ce qui donne

```
Hella @ o @ thr avant.  
Bonjour thr @ avant.  
Bonjour.
```

Notez que nous aurions pu également écrit

```
uf de la /.@//
```

et laissé à cela; ce qui maintient le bouclage jusqu'à ce qu'il a essayé de faire la substitution, mais n'a pas réussi à trouver des modèles correspondants ". @". La commande S serait alors définir le registre de condition à FALSE et la boucle s'arrêterait. Notez que cela entraînerait encore une répétition de la boucle UF que la version avec la commande T en elle.

La version finale de la commande jusqu'à l'UE (Jusqu'à Error). Cela exécutera sa liste de commandes jusqu'à ce qu'une erreur se produit. Notez que «incapable d'accéder au fichier" et "aucun texte modifié" les erreurs ne sont pas considérés dans le contexte de UE; deux d'entre eux peuvent être détectés en vérifiant la valeur du registre de condition, et donc ne sont pas considérés comme des erreurs «authentiques». Nous ne dirons pas plus sur la commande UE dans ce guide.

Le U toutes les commandes fonctionnent de la même manière que la commande globale: la balise <liste de commandes> sont copiés dans une mémoire tampon cachée et le tampon est alors exécuté. En raison de ce processus de copie, vous devez être prudent lors de l'utilisation des directives de flux dans les commandes U. Pour plus de détails, voir la section sur les directives Stream.

Nombre registres

Afin d'effectuer des opérations arithmétiques simples, FRED vous permet de définir le numéro des registres. Chaque registre de numéro a un nom qui respecte les restrictions sur les noms de tampons et les noms d'étiquettes.

registres numériques sont manipulés à l'aide de la commande N. Cette commande a la forme

N (<nom>) <option>

où <nom> est le nom du registre de nombre et <option> indique ce que vous voulez faire avec le registre. Certaines des options possibles sont énumérés ci-dessous.

: nnn

assigne le numéro «nnn» au registre. Par exemple, "n (xx): 5" affecte une valeur de 5 à "xx".

+ nnn

ajoute "nnn" au registre. Par exemple, «n (i) +1" on ajoute à "i".

-NNN

soustraient "nnn" du registre.

* nnn

multiplie le registre par "nnn".

/ nnn

divise le registre par "nnn". Ceci est la division entière standard.

% nnn

donne la valeur du registre modulo "nnn".

<nnn

compare la valeur du registre à "nnn". Si cette valeur est inférieure à "nnn", le registre d'état est réglé sur TRUE; sinon, il est réglé sur FALSE.

> nnn

compare la valeur du registre à "nnn". Si cette valeur est supérieure à "nnn", le registre d'état est réglé sur TRUE; sinon, il est réglé sur FALSE.

= nnn

compare la valeur du registre à "nnn". Si les deux sont égaux, le registre d'état est réglé sur TRUE; sinon, il est réglé sur FALSE.

P

affiche le contenu du registre. Par exemple, «n (xx) p" affiche la valeur de "xx".

UNE

attribue le numéro de ligne de la ligne courante au registre. Par exemple, "\$ n (fin) un" ensembles "fin" du nombre de lignes dans la mémoire tampon. Notez que si vous spécifiez une adresse unique pour la Une option de la commande N, le pointeur de la ligne en cours "." est fixé à cette adresse. Si vous spécifiez deux adresses, le pointeur de la ligne actuelle est réglée à la seconde adresse et le registre de numéro est réglé sur le numéro de ligne de la première adresse. Cette fonctionnalité est parfois utile lorsque vous essayez d'obtenir les numéros de ligne au début et la fin d'une partie de texte.

Les registres numériques peuvent être utilisés pour effectuer des opérations arithmétiques simples entier. Ils peuvent également être utilisés pour stocker des informations qui seront utilisées par la suite dans le programme.

En plus des registres de nombres normaux, il existe un registre de numéro spécial qui est représenté par le symbole "#". Ceci est connu comme le "count" enregistrer. La valeur du registre de comptage peut être imprimé avec la commande

#

Le registre de comptage est fixée par un certain nombre de commandes. Par exemple, la commande S définit le registre de comptage du nombre de substitutions qu'il fait. Ainsi la commande

s /./&/ #

va imprimer le nombre de caractères dans la ligne actuelle. Pourquoi? "S /./&/" change chaque caractère dans la ligne sur elle-même (rappelez-vous que "&" sur le côté droit d'une commande S pour une chaîne qui correspond le modèle de substitution). Comme il y a une substitution pour chaque caractère de la ligne, "#" est réglé sur le nombre de caractères sur la ligne. Si vous avez voulu attribuer ce numéro à un registre, on pourrait dire

n (xx): #

Vous pouvez également utiliser le symbole «\$» comme le numéro de la dernière ligne d'un tampon. Ainsi

n (fin): \$
\$ N (fin) un

à la fois attribuer le numéro de ligne de la dernière ligne dans le tampon au nombre registre «fin». La différence est que la première forme ne change pas la valeur du pointeur de la ligne en cours "." tandis que les seconds ensembles de forme "." à la dernière ligne dans le tampon.

Ci-dessous, nous montrons une commande utile qui utilise le numéro registres.

g / ^ / s /./&/ n (xx): # n (xx)> 72 jf p

Cette commande imprime chaque ligne dans le tampon courant qui contient plus de soixante-douze caractères. Le "g / ^ /" veille à ce que la liste des commandes sera exécuté pour chaque ligne dans le tampon. La commande S attribue le nombre de caractères dans la ligne du registre de comptage et le "n (xx): #" enregistre ce numéro dans "xx". La commande suivante teste la valeur de "xx". Si "xx" est pas supérieur à 72, le registre de condition est défini FALSE et le reste de la ligne est ignoré; autrement, le registre de condition est TRUE, la JF ne saute pas, et la ligne est imprimée.

Directives Stream

Lorsque vous utilisez FRED en mode interactif, FRED obtient habituellement tout son entrée à partir du terminal. Cette entrée peut être considéré comme un *flux* de caractères qui FRED interprète comme des

commandes, tapé dans le texte, et ainsi de suite. Une directive de flux est un moyen simple de dire FRED pour obtenir son entrée temporairement d'une source différente.

L'exemple le plus simple d'une directive de flux est "\ N". La construction "\ N (<nom>)" indique FRED pour obtenir son entrée temporairement à partir du numéro registre <name>. Par exemple,

```
n (vv): 5
jm / La valeur de "vv" est \ n (vv) ./
```

serait imprimer

La valeur de "v" est 5.

Lorsque FRED voit le "\ N", il sait qu'il est de ramasser l'entrée d'un registre de numéro plutôt que le terminal. Il va au registre, trouve qu'il contient un 5 et, par conséquent utilise le 5 où le "\ n (v)" a été trouvée. On pourrait aussi dire quelque chose comme

```
une
\ N (vv)
\F
```

Encore une fois, quand FRED voit le "\ N", il prend l'entrée du registre de numéro plutôt que du terminal. Les lignes ci-dessus seraient ajouter le numéro 5 au tampon courant.

Une autre directive de flux est simple "\ W". Lorsque FRED voit ce symbole, il le remplace par le nom du fichier qui est associé avec le tampon courant. Ceci est très commode, en particulier dans les commandes du système. Par exemple,

```
! Pascals \ W
```

sera envoyé à TSS par FRED en raison de la "!" sur le devant. Avant FRED envoie au large de la commande, il remplacera le "\ W" avec le nom de fichier qui est associé avec le tampon courant. Ainsi, la commande ci-dessus peut être utilisé pour soumettre dès qu'il est édité un fichier source Pascal pour la compilation. Cela peut économiser une quantité importante de dactylographie, surtout si vous travaillez avec des fichiers qui sont profondément dans une structure de catalogue.

La directive de flux le plus couramment utilisé est probablement "\ B". Ceci indique FRED pour obtenir son entrée à partir d'un tampon existant plutôt que le terminal. Par exemple, supposons que le tampon "a" contient le texte

```
s / moléculaire / atomique / p
```

taper

```
\ B (a)
```

a exactement le même effet que de taper "s / moléculaire / atomique / p". Lorsque FRED voit le "\ B", il va tout simplement et obtient son entrée dans la mémoire tampon plutôt que le terminal. Ainsi

```
une
\ B (a)
\F
```

serait ajouter le S et P commandes au tampon courant. Taper "\ b (a)" comme une commande remplacerait "moléculaire" avec "atomique" dans la ligne courante.

Il existe plusieurs avantages évidents à l'aide de l'instruction "\ B". Il est généralement beaucoup plus courte à taper que l'intégralité du contenu d'un tampon, et les économies de temps peut ajouter si vous devez exécuter la même instruction à plusieurs reprises. En outre, vous pouvez toujours modifier le contenu d'un tampon pour assurer qu'il n'y a pas d'erreur de frappe. Si vous tapez dans les modèles complexes ou des séquences complexes d'instructions, cela peut être extrêmement important - si vous faites une erreur en tapant une commande dans une mémoire tampon, vous pouvez toujours le fixer; si vous faites une erreur en tapant une

commande directement à FRED, vous pouvez encrasser les choses avant que vous ayez une chance de vous corriger.

La façon la plus simple pour exécuter un programme de tampon est avec "\ B". Tout ce que vous avez à faire est de taper vos commandes dans un tampon (par exemple "buff"), puis exécuter le tampon en disant "\ B (buff)" comme une commande FRED. Le temps qu'il faut pour taper les commandes dans le tampon est généralement plus que compensé par la réduction des erreurs ultérieures et dactylographie.

Compter le \ C de:

Une directive de flux comme "\ B" ou "\ N" est un *caractère unique* en ce qui concerne FRED. Certes, vous tapez une directive de flux comme un antislash suivi d'une lettre; Cependant, dès que FRED voit ce genre de combinaison, il se convertit en une représentation interne spéciale qui est un caractère unique.

Il est important de se rappeler quand vous écrivez des tampons, parce que tôt ou tard, vous voulez placer une construction de directive de flux dans votre tampon. Supposons, par exemple, que vous voulez mettre une ligne blanche en face de chaque ligne qui commence par un caractère "%", et ensuite vous voulez supprimer le "%". Cela prendra une commande I pour insérer la ligne blanche, puis un «s /% //" pour supprimer le "%". Vous commencez à taper votre tampon avec

```
une
/ ^% / I
```

```
\F
```

La Une commande commence à ajouter du texte à la mémoire tampon que vous créez. Vous tapez la commande I, la ligne blanche, et le "\ F", qui met fin à l'entrée de la commande I. Cependant, FRED pense que vous saisissez dans le "\ F" pour mettre fin à l'entrée de commande A! Comment obtenez-vous FRED se rendre compte que le "\ F" fait partie de votre texte, pas une indication que vous avez terminé la saisie de texte?

Vous vous souviendrez que le caractère "\ C" a été utilisé pour éteindre les significations particulières pour les caractères dans FRED. Dans un motif, par exemple, "\ C \$" représente le caractère "\$", au lieu de la fin de la ligne. De la même manière, en mettant un "\ C" devant "\ F" indique FRED que vous voulez traiter "\ F" comme un caractère ordinaire, non pas comme le caractère spécial qui indique la fin de la saisie de texte. Ainsi vous taperez

```
une
/ ^% / I

\ C \ F
+ 1s /% //
\F
```

Si vous listez maintenant le tampon, il contiendra

```
/ ^% / I

\F
+ 1s /% //
```

ce qui est exactement ce que vous voulez. (Notez que vous ne seriez pas obtenir les mêmes résultats en tapant "\\ F". "\\ F" vous donne un caractère antislash suivi d'un le "\ F" caractère unique "F", non.)

Le même principe est applicable lorsque vous souhaitez entrer d'autres constructions de directive de flux dans une mémoire tampon. Par exemple, supposons que vous voulez calculer le nombre de lignes complètement vides dans le tampon "0". Vous pouvez le faire avec le programme

```
b0 g / ^ * $ /
n (xx): #
jm, il y a \ N (xx) des lignes vides dans le tampon .;
```

(Rappelez-vous que la commande G définit le registre de comptage du nombre de ligne associée par le modèle.)

Si vous avez tapé la commande ci-dessus comme JM

```
jm, il y a \ n (xx) des lignes vides dans le tampon .;
```

FRED remplacerait le "\ N (xx)" dès qu'il a vu. Si "xx" contenait le numéro 5 dire, votre tampon contiendra

```
jm / Il y a 5 lignes vides dans le buffer./
```

et vous souhaitez toujours obtenir ce message lorsque le programme a été exécuté. Qu'est-ce que vous voulez dans la mémoire tampon est un caractère "\ N". Ainsi, lorsque vous tapez dans vos commandes vous devez taper

```
jm / Il est \ C \ N (xx) Les lignes vides dans le buffer./
```

Le "\ C \ N" se transforme en un seul caractère "\ N", qui est ce que vous voulez.

Cela est d'autant relativement simple, mais il est facile d'oublier quand vous ne faites pas attention. Les choses deviennent un peu plus délicat lorsque vous utilisez les commandes globales et Jusqu'à. Rappelez-vous que ces commandes copient leurs listes de commandes dans un tampon privé, puis exécuter les commandes dans ce tampon. Par exemple, supposons que vous vouliez faire une déclaration mondiale qui non seulement trouver des lignes vides dans un tampon, mais diriez-vous aux numéros de ligne des lignes vides. Au début, vous pourriez penser que vous pourriez écrire cela comme

```
g / ^ * $ / n (bl) un jm: Ligne \ N (bl) est vide .:
```

Cette commande de G n'exécutera sa liste de commandes sur des lignes vides. La commande N attribue le numéro de ligne de chaque ligne vide au nombre registre "bl" et la commande JM imprime un message. Toutefois, si vous essayez d'exécuter la commande ci-dessus, vous trouverez que vous obtenez beaucoup de messages de la forme

Ligne 0 est vide.

Que s'est-il passé? Lorsque la liste des commandes a été copié dans le tampon caché, le "\ N (bl)" a été *immédiatement* remplacé par la valeur du nombre registre "bl" (probablement zéro). Ainsi la commande JM qui a été exécuté de manière répétée était

```
jm: Ligne 0 est vide .:
```

ne pas

```
jm: Ligne \ N (bl) est vide .:
```

Pour faire passer le message de JM approprié dans la mémoire tampon de la commande globale, la commande doit avoir la forme

```
g / ^ * $ / n (bl) un jm: Ligne \ C \ N (bl) est vide .:
```

Cela donnera un certain nombre de différents messages comme

Ligne 34 est vide.

Ensuite, comment voulez-vous taper cette commande dans un tampon? Nous l'avons souligné ci-dessus que le typage "\ C \ N" tandis que le texte annexant vous donne juste un "\ N" dans le tampon. Pour entrer un "\ C" et un "\ N", vous avez besoin d'un "\ C" pour le "\ C" et un autre "\ C" pour le "\ N". Ainsi, si vous avez voulu entrer la commande appropriée dans un tampon, vous devez taper

```
g / ^ * $ / n (bl) un jm: Ligne \ c \ c \ c \ n (bl) est vide .:
```

Si vous imprimez maintenant la ligne de la mémoire tampon, FRED montrera

g / ^ * \$ / n (bl) un jm: Ligne \ C \ N (bl) est vide .:

Ceci est un bon moment pour faire quelques observations sur les cas de lettres dans des séquences d'échappement. Vous pouvez saisir le caractère "\ C" soit avec un boîtier supérieur ou inférieur "C". Cependant, dès que FRED voit soit "\ c" ou "\ C", il va le convertir à sa représentation interne spéciale; à partir de ce moment-là, le caractère sera toujours imprimé comme "\ C" depuis FRED affiche toujours ces caractères spéciaux avec des lettres majuscules. Le même principe est valable pour tous les autres caractères de la directive de flux.

Obtenir le bon nombre de caractères "\ C" dans vos commandes n'a pas à être difficile; si vous pensez que les choses attentivement, vous devriez avoir aucun problème. Cependant, nous devons vous avertir que vous pouvez parfois besoin de beaucoup de caractères "\ C" pour faire les choses. Pour terminer cette section, nous illustrons un tel problème.

Supposons que vous avez trouvé un essai prêt pour TFing. Chaque paragraphe dans votre essai est actuellement en retrait de cinq espaces. Vous souhaitez supprimer ces cinq blancs et plutôt mettre une commande tiret temporaire ".ti +5" au début de chaque paragraphe. Pour ce faire d'un paragraphe, vous pouvez utiliser la commande

```
s / ^ /.ti +5 \ C
/
```

Cela remplace les cinq blancs au début de la ligne avec ".ti +5" suivie d'un retour chariot. Le "\ C" est nécessaire devant le retour de chariot de telle sorte que FRED sait qu'il ya plus à venir dans la commande. Si vous alliez mettre cela dans une commande G, vous auriez à taper

```
g / ^ / s //. ti +5 \ C \ C \ C
/
```

Le premier "\ C \ C" vous donne le caractère "\ C" vous avez besoin dans la commande S. Ensuite, vous voulez un retour chariot, mais vous avez besoin d'un "\ C" en face de cela aussi, encore une fois pour dire FRED qu'il ya plus à venir. Enfin, si vous tapiez ceci dans un tampon que vous auriez à taper

```
g / ^ / s //. ti +5 \ c \ c \ c \ c \ c
/
```

Pour chaque "\ C" que vous voulez dans la commande, vous devez taper deux lors de l'ajout de la commande à un tampon.

Si vous trouvez que vous avez besoin trop de caractères "\ C", il est fort possible que vous faites les choses difficiles pour vous-même. Par exemple, nous aurions pu éviter certains des problèmes ci-dessus en utilisant la commande

```
* s / ^ /.ti +5 \ C
/
```

au lieu d'une commande globale. Avec un peu de réflexion, vous pouvez souvent éliminer beaucoup de problèmes "\ C" en allant sur les choses d'une manière différente.

D'autres directives Stream:

Il existe plusieurs autres directives de flux que vous pourriez trouver utiles lors de l'écriture des programmes tampons.

"\ L" est similaire à "\ B» en ce qu'il met le contenu d'un tampon dans le flux d'entrée. La différence est que tous les caractères de la mémoire tampon est traitée comme si elle est précédée d'un "\ C". Dans l'un de nos exemples, nous allons montrer comment cela peut être utilisé.

"\ S" est similaire à "\ L". La différence est que chaque caractère de nouvelle ligne dans le tampon est retiré avant que le tampon est placé dans le flux d'entrée.

La construction "\ R" est à portée de main quand un programme tampon a besoin d'entrée de l'utilisateur. Lorsque FRED voit un caractère "\ R", il détourne le flux d'entrée à la borne pour obtenir une ligne d'entrée. Aucune directive de flux tapés sur cette ligne sont efficaces. Le caractère de nouvelle ligne à la fin de la ligne d'entrée est ignoré. Une façon très simple à utiliser cette fonction est indiquée dans le tampon suivant.

```
jp / Quel fichier voulez-vous que je lise? /  
br * da \ R \ F  
b (fichier) r \ S (r)
```

Celui-ci utilise une commande de JP pour demander à l'utilisateur un nom de fichier. Buffer "r" est autorisé par un "* d" et la réponse de l'utilisateur est ajouté à ce tampon. La déclaration finale passe à tampon "fichier" et lit dans le fichier désiré en utilisant le "\ S (r)" construction pour obtenir le nom du fichier à partir du tampon "r". On aurait aussi pu utiliser "\ B (r)" ou "\ L (r)" dans ce cas, puisque les trois directives de flux auraient le même effet.

La directive flux final, nous allons discuter est "\ E". La commande E est utilisée pour définir et nommer un motif dans FRED; par exemple,

```
e (d) / ^ [0123456789] /
```

définit un motif appelé "d". (Noms de motifs suivent les mêmes règles que les noms de tampons et les noms de l'étiquette.) Cette tendance va correspondre à toute ligne qui commence par un chiffre (rappelons que le modèle comportant un certain nombre de caractères entre crochets correspond à l'un des caractères dans les parenthèses) . Une fois que le modèle a été défini dans une commande E, vous pouvez l'utiliser à l'intérieur des motifs de la directive de flux "\ E (<nom>)". Par exemple

```
/ \ E (d) / d
```

va supprimer la première ligne, il constate que commence par un chiffre. Notez que le "\ E" directive de flux est applicable uniquement à l'intérieur des motifs FRED.

Il y a trois ou quatre autres directives de flux qui sont reconnus par FRED, mais nous ne les discuter ici. Pour plus de détails, voir le manuel de référence FRED [2].

commentaires

Il est toujours une bonne idée de mettre des commentaires dans vos programmes, si les programmes sont écrits en FRED ou dans une autre langue. Commentaires à FRED sont écrits en utilisant le caractère double citation. Lorsque FRED voit une double citation dans toute commande autre qu'un message JM, JP, ou JE, il ignore tout de la double citation à la fin de la ligne. Par exemple,

```
"Ceci est un commentaire  
* D "cela efface le tampon  
"Ce * d ne fait rien
```

Dans un programme de tampon, vous voulez bien souvent d'aller à une ligne de texte sans impression que sortie ligne. Cela peut être fait en utilisant la double citation avec un numéro de ligne. Par exemple,

```
1"
```

va à la première ligne de texte, mais n'imprime pas dehors. Si vous venez de taper le 1 sans les guillemets, la première ligne sera imprimée. Une autre façon de le faire est d'utiliser la commande Z (adresse Zap). Par exemple,

```
3z
```

définit le pointeur de la ligne courante à la ligne 3 sans imprimer quoi que ce soit.

Lire les listes

Parfois, un tampon FRED veut lire un fichier, mais ne sait pas le nom ou l'emplacement du fichier. Pour gérer les situations, FRED vous permet de spécifier une liste de noms de fichiers possibles sur la commande R, comme dans

```
r fichier1, fichier2, fichier3
```

Lorsque FRED trouve une commande de R comme cela, il commence par essayer de lire le premier fichier dans la liste. Si cette lecture est réussie, FRED passe à la commande suivante après la R. Cependant, si la lecture échoue parce que le fichier ne peut pas être trouvé (ou parce qu'il ya une erreur de syntaxe dans le nom de fichier), FRED essaie de lire le second fichier dans la liste. FRED continue d'essayer fichier après fichier dans la liste jusqu'à ce qu'il trouve un fichier ou il arrive à la fin de la liste.

Si l'une des opérations de lecture fonctionne, le registre de condition sera réglé sur TRUE et le registre de comptage du nombre de blocs lus. Si aucune des opérations de lecture fonctionne, le registre de condition est définie sur FALSE.

Notez que FRED ne tente un nouveau fichier sur la liste si le fichier précédent n'a pas pu être trouvé ou son nom avait une erreur de syntaxe. S'il y a une autre erreur dans l'opération de lecture (par exemple, les autorisations refusées), FRED ne cherche pas d'autres fichiers. FRED établit simplement le registre de condition à FALSE et passe à la commande après la R.

Comme un cas particulier, vous pouvez mettre une virgule de fuite à la fin de la liste des fichiers, comme dans

```
r fichier1, fichier2, fichier3,
```

Ceci indique FRED pour régler le registre de condition à TRUE, même s'il ne peut pas trouver les fichiers sur la liste. Si FRED va droit dans la liste et ne trouve pas les fichiers, le registre de condition est TRUE et le registre de comptage est mis à -1. Rappelez-vous que si une opération de lecture échoue pour une raison autre que «fichier non trouvé» ou «fichier erreur nom de syntaxe", FRED jamais arrive à la fin de la liste et le registre de condition sera définie sur FALSE.

A titre d'exemple de la façon dont cela pourrait être utilisé, nous allons revenir à un tampon qui nous avons écrit au début de ce guide. Nous avons d'abord écrit comme

```
r / file1 j (go) t
r / file2 j (go) t
jm: Vous ne pouvez pas lire les deux / file1 ou / file2 .:
q
@(aller)
"Plus les commandes
```

Nous pouvons maintenant raccourcir ce à

```
r / file1, / file2
jt je: Vous ne pouvez pas lire les deux / file1 ou / file2 .:
```

Comme autre exemple, supposons qu'un tampon particulier permet aux utilisateurs d'avoir un fichier nommé "options" contenant FRED commandes que les options définies pour le programme de la mémoire tampon. Ce fichier peut être stocké dans un fichier rapide d'accès nommé "/ options", un fichier temporaire nommé "options", ou un fichier dans le catalogue "/ fred" (/ fred / options). Pour exécuter les commandes telles, vous pourriez écrire

```
b (options) * dr / options, options, / fred / options,
jt je: Erreur fichier d'options de lecture:
\ B (options)
"Plus instructions
```

Non seulement cela vous donne la possibilité de rechercher le fichier dans un certain nombre d'endroits, mais il vous donne aussi la possibilité d'indiquer l'ordre dans lequel FRED doit rechercher les fichiers.

Alors que nous examinons ces deux tampons, on pourrait dire un peu plus sur la façon de produire des messages d'erreur informatifs. Si nous avons pris la ligne

```
jt je: Erreur fichier d'options de lecture:
```

et changé à

```
jt jp>Error options de lecture fichier: 'y je
```

nous pouvons utiliser la commande Y pour nous donner un rapport plus détaillé de ce que l'erreur réelle était. En regardant le programme précédent, nous aurions écrit

```
r / file1, / file2
jt jp \ 'W:' y je
```

Cette affiche le nom du dernier fichier FRED a essayé de lire, suivie par quelque Y fournit comme la raison de l'erreur.

Cette technique imprime toujours le nom du *dernier* fichier FRED essayé de lire dans la liste de lecture, vous pouvez ajuster légèrement la liste de lecture pour fournir une erreur plus informatif. Par exemple, supposons que la personne qui utilise le programme est censé entrer le nom d'un fichier et le fichier peut être dans un fichier d'accès rapide ou sous le catalogue `"/ fred"`. Vous pourriez écrire

```
jp? S'il vous plaît entrez le nom de fichier :?
b (fichier) a
\ R
\F
b (en) r \ S (fichier), / fred / \ S (fichier), \ S (fichier),
jt jp? Erreur sur \ W :? y je
```

Cette première recherche le fichier sous le nom que l'utilisateur a donné, puis regarde sous `"/ fred"`. Si le fichier est pas sous le catalogue, il essaie à nouveau sous le nom d'origine. Bien sûr, le fichier ne sera pas encore là, mais quand le message d'erreur est imprimé, il donnera le nom que l'utilisateur a tapé dans (qui sera plus facile à comprendre).

Exécution FRED Tampons

Il y a deux façons simples pour exécuter un tampon FRED. Si vous êtes déjà dans FRED et vous avez la mémoire tampon disponible, vous pouvez exécuter les commandes juste en tapant

```
\ B (nom de tampon)
```

au niveau de la commande. FRED va chercher les commandes à partir de la mémoire tampon spécifiée et les exécuter un par un. Quand il est à court de commandes, il reviendra à la prise des commandes directement depuis le terminal (à condition bien sûr que le tampon n'a rien drastique comme arrêter de fumer FRED alors qu'il était en cours d'exécution).

Si vous êtes au niveau du système et que vous avez un programme de tampon stocké dans un fichier, vous pouvez exécuter le programme avec une commande de la forme

```
fred filename arg1 arg2 arg3 ...
```

Lorsque FRED voit ce genre de ligne de commande, il lit le contenu du fichier spécifié dans un tampon appelé `"b (.)"`. Les arguments qui sont donnés sur la ligne de commande sont placés l'un par ligne dans le tampon `"b (0)"`. Votre programme de tampon peut obtenir ces arguments de `«b (0)»` de son exécution. En tant que service à l'utilisateur, FRED met également votre identifiant dans un tampon appelé `"b (u)"`, la date dans un tampon appelé `"b (d)"`, et l'heure actuelle dans un tampon appelé `"b (t)"`.

Vous ne devez pas spécifier le nom de fichier complet du fichier qui contient le tampon. FRED va essayer de lire le nom de fichier donné en premier lieu; si cela ne fonctionne pas, il va chercher sous votre userid / fred / filename; et ce qui échoue aussi, il va regarder sous

```
bibliothèque / fred / filename
      et
fred / filename
```

En substance, il place le nom du fichier dans un tampon que nous appellerons "f" et effectue ensuite la séquence de commandes

```
b (.)
r \ S (f), \ S (u) / fred / \ S (f), bibliothèque / fred / \ S (f) / , fred / \ S (f)
jt je /? n'a pas pu accéder \ B (f) /
@ (A) "et ainsi de suite
```

Notez comment nous avons utilisé le userid qui est stocké dans un tampon "b (u)" dans la construction "\ S (u)".

Il faut souligner ici que FRED exécutera votre fichier init FRED avant qu'il ne commence l'exécution d'un programme de tampon. Nous dirons plus sur les fichiers d'init lorsque nous arriverons à nos exemples de tampons.

Une fois que FRED a exécuté un programme appelé de cette façon, il va quitter avec QQ.

options de

La plupart des programmes de tampon exigent que certaines options FRED être activées et certains être éteints. Parce que les tampons ont tendance à être utilisés pour des processus complexes, ils utilisent souvent des options que l'utilisateur normal ne pas activé par défaut. Pour cette raison, vos tampons auront souvent à mettre en place leurs propres environnements d'options avant de commencer l'exécution.

Dans les exemples qui suivent cette section, nous allons toujours supposer que FRED commence avec les options par défaut standard définies. Quand nous avons besoin de changer ces options, nous allons le faire explicitement.

Si vous écrivez un tampon qui peut être utilisé par d'autres utilisateurs en dehors de vous-même, vous devez garder à l'esprit qu'ils peuvent avoir des fichiers d'initialisation qui définissent les différentes options par défaut que votre propre. En raison de cela, vous aurez généralement à définir toutes les options dont vous avez besoin explicitement au début de votre tampon.

Les directives de flux "\ C" et "\ O" peuvent être utilisés dans certains cas pour forcer des options spéciales est désactivé ou activé. A "\ C" devant un caractère spécial de motif signifie que le caractère est à prendre littéralement, indépendamment de toute spéciale qui signifie qu'il pourrait avoir en raison des options. Par exemple, supposons que vous souhaitez tester un "{" en ligne. Si l'utilisateur a spécifié le "{O + S" option, la commande

```
t / {/
```

recevra une erreur de syntaxe, car le caractère spécial "{" est utilisé de manière incorrecte dans le motif. Pour contourner cela, vous pourriez dire

```
t / \ c {/
```

Cela permettra de tester un "{" caractère indépendamment du fait que le "O + S {" ou "OS {" option est en vigueur. Le "\ C" indique FRED pour prendre la "{" littéralement.

De la même manière, "\ O" en face d'un personnage raconte FRED d'interpréter ce personnage avec toute spéciale qui signifie qu'il pourrait avoir dans ce contexte. Par exemple, la commande

s / abc / && /

tournera "abc" dans "abcabc" si le "O + S &" option est en vigueur; cependant, il se révélera "abc" en deux esperluette si l'option est pas en vigueur. A "\ O" en face d'un personnage tourne sur son sens particulier, quelles que soient les options de voies sont actuellement définies. Ainsi

s / abc / \ O & \ O & /

tournera "abc" dans "abcabc" si "O + S &" ou "OS et" est en vigueur. Ainsi, "\ C" et "\ O" fournissent un autre moyen de contourner les différences dans les paramètres des options.

Il y a une option importante que nous devrions parler avant que nous commençons nos exemples. L'option du moniteur est activé avec la commande

O + M

Cette option indique FRED pour surveiller tous les travaux effectués au cours de l'exécution des tampons. Cela comprend le travail effectué dans les tampons cachés utilisés par les commandes G et U. De cette façon, FRED fournit une trace de ce que les commandes ont été exécutées dans la mémoire tampon.

La sortie du moniteur est imprimé à la borne et est un caractère par caractère reproduction des commandes et des textes utilisés par FRED lors de l'exécution de la mémoire tampon. Commentaires et commandes sauté par sauts ne sont pas imprimés. D'une manière générale, la sortie du moniteur est imprimé avec de nombreuses commandes sur la même ligne. Cela peut être un peu désordonné, mais il est également très utile lorsque vous essayez de déboguer vos programmes tampons.

La commande OM arrête la sortie Monitor.

Exemples de programmes Buffer

Nous allons maintenant donner un certain nombre d'exemples de programmes de tampons FRED. Les programmes traitent d'une grande variété d'applications, certains plus spécialisés que d'autres. Que ce soit ou non vous serez en mesure d'utiliser l'un de ces tampons dans votre propre travail, ils auront au moins démontrent comment FRED commandes peuvent être combinées pour effectuer des tâches spécifiques.

Notez que lorsque nous présentons un programme de tampon, nous allons montrer comment il devrait ressembler une fois qu'il est à l'intérieur du tampon. Lorsque vous annexant ces commandes à un tampon, ne pas oublier que vous devrez mettre "\ C" caractères en face de chaque directive de flux, nous montrons ici afin que tous les caractères spéciaux sont entrés correctement.

Exemple 1: Réduction d'un fichier de la carte

Le premier exemple traite d'un fichier qui a été créé par la lecture dans un jeu de cartes [4] contenant un programme Fortran qui doit être préparé pour une utilisation sur le système de partage des temps. Le problème est qu'il ya beaucoup de blancs dans le fichier. En fait, chaque ligne a 80 caractères, quelle que soit la longueur de l'instruction Fortran. Nous voulons supprimer les espaces de fin et de vérifier que toute carte de continuation a une esperluette (&) dans la colonne 6.

Ce tampon est l'un qui peut être invoqué au niveau de la commande (qui est en réponse à l'invite "*") au moyen d'une commande comme

```
fred rétrécir infile outfile
```

où «psy» est le nom d'un fichier contenant le tampon suivant. "Infile" est le nom du fichier contenant le programme Fortran et "outfile" est le nom du fichier dans lequel le code source ratatinées doit être stocké. Si "outfile" est pas spécifié, le tampon utilisera le workfile "*" src" pour contenir le code ratatinées; si "infile" est pas spécifié non plus, le tampon suppose que le code non-rétréci est stocké dans "*" src". Rappelez-vous que le chargeur de bootstrap FRED place les arguments sur la ligne de commande sur des lignes séparées dans un

tampon "0". Pour cette raison, l'une des premières choses que le tampon suivant n'est de vérifier combien de lignes il y a dans le tampon "0" pour voir combien de fichiers ont été spécifiés sur la ligne de commande.

```
o { "Définir les options s + pour motif de marquage
o + option set Quitter q "
b0 "passer au tampon 0
n (xx): $ "affecter le nombre de lignes dans
          "Tampon" 0 "pour enregistrer" xx "
n (xx)> 1 "set code de condition à" true "si
          «Le nombre est supérieur à 1
j (a) t "saut à l'étiquette d'un si" true "
$ A * src
* src
\ F "append deux lignes avec" * src "après
          "La dernière ligne de la mémoire tampon
@ (A) "une étiquette
1m (f.in) "déplacer le nom du fichier d'entrée
          "Au tampon" f.in "
1m (f.out) "déplacer la ligne à côté de tampon" f.out "
* D "supprimer toutes les lignes restantes
          "Dans un tampon" 0 "
fichier d'entrée r \ S (f.in) "lire
jt y je //
g ~ / ^ c / s / ^ {.....} t [^] / t \ C \ C & /
          "Si aucun commentaire, le changement continue
          "Caractère &
* s / * $ // "supprimer les blancs à la fin
          "De chaque ligne
w \ S (f.out) "écrire fichier ratatinées
jt y je //
q "tout fait
```

La déclaration lue

```
r \ S (f.in)
```

suppose que le tampon "f.in" contient le nom d'un fichier qui doit être lu. Si la lecture est réussie, le registre de condition est réglé sur TRUE. Le JT sur la ligne suivante va sauter sur Y et le JE et l'exécution continuera à la commande G. D'autre part, s'il y avait des erreurs détectées lors de la tentative de lire (par exemple, essayer de lire un fichier inexistant) le registre de condition est définie sur FALSE. La commande Y sera alors exécuté pour indiquer à l'utilisateur ce qui a mal, et JE arrêtera toute exécution.

La déclaration

```
g ~ / ^ c / s / ^ {.....} t [^] / t \ C \ C & /
```

localise d'abord toutes les lignes qui ne commencent pas par «c» (commentaire déclarations). La commande S provoque alors des cinq caractères au début de la ligne (l'expression marquée {.....} t) suivi d'un caractère non-blanc pour être remplacé par exactement les mêmes cinq caractères suivis d'une esperluette. Le "\ C" doit apparaître en raison du rôle particulier de l'esperluette dans la substitution. Puisque vous voulez un caractère esperluette et non pas la chaîne qui correspond à la configuration de la commande S, vous devez avoir un "\ C &" lorsque la substitution est faite. Parce que la liste des commandes pour la commande G sont *joint* à un tampon caché, vous devez écrire "\ C \ C" pour mettre un seul "\ C" dans le tampon caché. Lorsque vous tapez cette commande dans la mémoire tampon, rappelez - vous que vous devez taper *quatre* caractères "\ C" pour obtenir deux sur.

Le «* s» dans la déclaration

```
* s / * $ //
```

signifie que la substitution doit être tentée à chaque ligne. Le motif "*" correspondra à toute chaîne de zéro ou plusieurs ébauches et le "\$" correspond à la valeur nulle (imaginaire) caractère avant le retour chariot à la fin de la ligne. Par conséquent, le motif "* \$" correspond à toute chaîne d'un ou plusieurs espaces à la fin

d'une ligne. La commande ci-dessus supprime ainsi tous les blancs à la fin d'une ligne en les remplaçant par rien. Une autre forme qui fonctionnera aussi est "1, \$ s / * \$ //".

Vous pouvez vérifier que les blancs indésirables ont été retirés du fichier en faisant la substitution "/ s /% de de * /", l'affichage du tampon à l'aide de la commande "* p", puis changer le "%" retour à flans par moyen de la commande "* s /% / /". Ici, nous avons utilisé le "%", car il est un personnage qui est rarement utilisé en Fortran code source. Vous pouvez utiliser tout autre caractère qui ne figure pas déjà dans votre tampon pour ce test. Lors du changement de retour à des blancs, toutes les occurrences de «%», qui aurait déjà été présent seront remplacés par des blancs. Si vous voulez supprimer tous les blancs (! Bonne douleur) de la mémoire tampon, essayez ceci: "* s / //".

Vous vous demandez probablement comment vous pouvez dire si un caractère ou une chaîne particulière est présente dans la mémoire tampon. La déclaration

```
g /% / p
```

permet d'imprimer toutes les lignes contenant un "%". De même, la déclaration

```
g / CALL / p
```

imprime chaque ligne qui contient la chaîne "CALL". Pour savoir comment de nombreux cas de la chaîne "CALL" il y a, vous pouvez utiliser

```
* S / CALL / & / #
```

Comme nous l'avons mentionné précédemment, la commande S définit le registre de comptage du nombre de substitutions effectuées avec succès.

La déclaration d'écriture "w \ S (f.out)", écrit le contenu du tampon "0" dans le fichier dont le nom est dans le tampon "f.out". Si une erreur est détectée par écrit, ou si un fichier doit être créé, la commande Y sera exécutée pour dire quel était le problème et l'exécution arrêtera.

Il y a une variation sur le tampon ci-dessus, qui réduit le nombre d'états FRED pour cet exemple, au détriment d'avoir un algorithme légèrement plus long. Cette variation omet simplement les deux commandes et le N «j (a) t" au début du programme, et ajoute automatiquement deux lignes contenant "* src" à "b (0)". Le résultat final est le même.

Il y a une deuxième variante de cet exemple pour les utilisateurs qui préfèrent utiliser le fichier d'entrée que le fichier de sortie par défaut si un seul nom de fichier a été spécifié. Cela se fait en remplaçant les trois lignes immédiatement après la «j (a) t" avec les commandes

```
$ A * src  
\F  
1k (.w) za (.w)
```

Ce joint une copie de la première ligne de "b (0)" à la fin de "b (0)". Ainsi, si un seul nom de fichier a été spécifié sur la ligne de commande, ce fichier sera utilisé pour l'entrée et la sortie.

Vous pouvez constater que l'une des conventions ci-dessus d'avoir trois formes différentes de la même commande est utile dans d'autres contextes. Bien sûr, les commandes G et S peuvent être remplacés par tous les états que vous souhaitez exécuter.

Exemple 2: Un fichier mémo

Dans cet exemple, nous allons montrer comment créer un fichier mémo. Chaque message est entré au début du fichier de sorte que la note la plus récente vient en premier (dernier entré, premier sorti). Chaque mémo porte la date et l'heure auxquelles il a été saisi. Cet ensemble d'instructions peut être tapé directement depuis votre terminal.

```

b (.w) * dr / mémos
0a ***** \ S (t) \ S (d)
.
. votre message
.
\F
w / mémos

```

La deuxième ligne commence à insérer du texte au début du tampon "b (.w)" (ie, après la ligne zéro, avant une ligne). La ligne de la nouvelle note d'ouverture contient une chaîne d'astérisques comme une décoration et la date et l'heure. Notez comment nous avons utilisé les tampons "b (t)" et "b (d)" qui sont fournis par le bootstrap FRED; "B (t)" contient le temps que FRED a été saisi et "b (d)" contient la date. Après cette ligne d'ouverture, vous pouvez entrer un certain nombre de lignes de texte comme un message mémo. Entrée arrête lorsque vous tapez un "\ F".

Il serait commode si nous pouvions faire ces étapes pour avoir lieu sans avoir à taper chacun d'eux à chaque fois. Ce que nous allons faire est de préparer un tampon appelé "m" qui contiendra les instructions nécessaires pour faire le travail. Pour ajouter une note à un fichier de notes ou de créer un fichier avec une note de service, tout ce que nous devons faire est de taper

```
\ B (m)
```

Cela entraînera tous les FRED commandes dans le tampon "m" pour être exécuté. Notre tampon "m" contiendra les commandes suivantes.

```

b (.w) * dr / mémos
0a ***** \ S (t) \ S (d)
\F
uf .a \ C \ R \ C
\ F .t ./
w / mémos

```

Ce tampon commence par la lecture du fichier "/" mémos" (qui ne doit pas exister) dans le tampon vide ".w". Une ligne d'ouverture est placée au début de ce tampon pour marquer le début de la nouvelle note de service.

La commande UF qui commence dans la ligne quatre porte à la ligne de cinq en raison de la "\ C" devant le retour chariot; le "\ C" indique que le retour chariot est pas la fin de la boucle jusqu'à ce que, il est juste un caractère qui est utilisé dans la boucle. La commande UF commence par la création d' un tampon caché et *annexant* toutes les instructions qui suivent ce jusqu'à la prochaine retour chariot qui est pas précédé par un "\ C". Ainsi , la dernière instruction dans la mémoire tampon jusqu'à ce que est ".t ./". Le "\ C" est nécessaire avant que le "\ R" pour les raisons habituelles lors de l' utilisation des directives de flux dans G et U commandes. Le tampon caché pour la commande UF va donc contenir

```

un \ R
\ F .t ./

```

La commande T définit le registre de condition à TRUE si tous les caractères ont été saisis comme entrée pour le "\ R". Si un retour chariot était la seule chose entré, le registre de condition est définie sur FALSE et la boucle se termine UF. Ainsi, pour indiquer la fin de votre mémo, tout ce que vous avez à faire est de taper dans une ligne nulle (ie juste un retour chariot).

Exemple 3: A Write Buffer

L'une des expériences les plus malheureux des utilisateurs d'ordinateurs est accidentellement écriture sur un fichier important avec les ordures que vous ne l'avez pas l'intention d'y mettre. Une façon d'éviter cela est d'adopter une convention par laquelle chaque fichier contient le chemin sous lequel il doit être stocké. Une convention qui a été trouvé commode est d'avoir la première ou la deuxième ligne de chaque fichier prendre la forme

```
commenter === === pathname
```

Les signes d'égalité servent à compenser le chemin d'accès qui identifie le fichier dans lequel le tampon doit être écrite. Le "comment" peut être une chaîne qui identifie un commentaire dans le langage de programmation utilisé (par exemple, "C" en Fortran, "/" * en B, ou «.cc.» Dans TF). Par exemple,

```
C Root Finder === / FORTRAN / NEWTON ===  
"Un tampon d'écriture === / FRED / WRITE ===
```

Supposons que vous avez édité un tampon et que vous souhaitez écrire dans un fichier. Nous allons créer un tampon "w" qui sera exécutée en entrant "\ B (w)". S'il n'y a pas de ligne du formulaire standard ci-dessus, le message «incapable d'écrire" est imprimé sur le terminal. S'il y a une ligne de ce formulaire, la date et l'heure sont substitués après le deuxième triple de signes égaux et le tampon sont écrites dans le fichier nommé. S'il y a une erreur d'écriture ou si un fichier doit être créé, le message "Erreur d'écriture sur <filename>" est donnée. Sinon, le message "écriture réussie sur <filename>" est imprimé. Voici tampon "w".

```
o + s {  
e (.w) / {^. * ===} 1 * {[^] [^] *} 2 * ===. * $ /  
n (ici) un "enregistrer le numéro de la ligne courante  
* T / \ E (.w) / "test pour la présence de la ligne  
jt je / incapable d'écrire /  
s / \ E (.w) / 1 2 === \ S (t) \ S (d) /  
                                "Insérer le temps à la  
                                " fin de la ligne  
k (fname) "copier la ligne b (fname)  
u1 b (fname) s / \ E (.w) / 2 /  
                                "Obtenir le nom de fichier seul  
\ N (ici) "" restaurer le numéro de la ligne courante  
wx \ B (fname)  
jt je / écrire erreur sur \ S (fname) /  
jm / écriture réussie sur \ S (fname) /
```

La première utilisation de l'expression ".w" est en ligne à quatre où l'on regarde pour la première ligne qui contient l'expression. Si le code de condition est pas TRUE, la ligne suivante affiche le message «incapable d'écrire". L'expression est utilisée à nouveau en ligne six, où tout suivant le deuxième triple signe égal est remplacé par le contenu des tampons "t" et "d". La ligne contenant le nom du fichier est copié dans "fname" avec la commande K.

La commande suivante met temporairement en mémoire tampon "fname" et fait une substitution qui réduit la ligne de commentaire complet en une ligne qui ne contient que le nom de fichier requis. Ceci est mis à l'intérieur d'une commande U1 à faire usage de l'une des caractéristiques les plus pratiques de la commande U. A la fin d'une boucle en U, le tampon courant est toujours placé à la mémoire tampon qui était en cours avant le début de la boucle, quel que soit le tampon était en cours à la fin de la boucle. En mettant le changement de tampon "fname" à l'intérieur d'une boucle en U, on peut manipuler le contenu de "fname" et toujours revenir au tampon courant appropriée lorsque nos manipulations sont terminées. De cette façon, nous pouvons laisser le tampon courant et retourner sans jamais avoir besoin de connaître le nom du tampon courant.

Une fois que nous avons le nom du fichier dans "fname", nous pouvons écrire dans ce fichier en utilisant la commande "wx \ B (fname)". WX est une simple variation de la commande W habituelle; la commande W n'imprime pas sur les statistiques d'écriture habituels (nombre de blocs, des lignes et des caractères écrits) quand il est exécuté dans un tampon, tandis que la commande WX N'imprimer ces statistiques.

La seule autre chose que nous devons noter sur le tampon ci-dessus est qu'il enregistre le numéro de la ligne actuelle du nombre registre «ici» et restaure ce numéro de ligne dès que les manipulations tampons sont finis.

Exemple 4: Init Tampons

Comme vous devenez plus à l'aise avec FRED, vous pouvez constater qu'il ya certaines choses que vous faites toujours à chaque fois que vous entrez dans le sous-système. Par exemple, vous pouvez définir certaines options qui ne sont pas par défaut standard ou vous pouvez lire dans les tampons utilitaires pratiques comme le tampon d'écriture, nous avons discuté dans le dernier exemple.

Vous pouvez avoir toutes ces choses pour vous automatiquement en créant ce qu'on appelle un *fichier init*. Chaque fois que FRED est appelé, soit avec la simple commande "FRED" ou une ligne de commande plus complexe de la forme

```
fred filename arg1 arg2 ...
```

l'éditeur recherche un fichier sous votre userid appelé "/fred/.init". Le contenu de ce fichier sera lu dans le tampon "b (f)" et seront exécutées avant que quelque chose se passe ailleurs. Une fois que cela a été fait, l'éditeur supprime tampon "b (f)". Ensuite, il charge le programme soit de tampon nécessaire ou commence à prendre des commandes directement depuis le terminal.

Un fichier init peut être aussi simple ou ambitieux que vous voulez. Par exemple, ce qui suit est un tampon parfaitement bien init.

```
o + s {([|
o + p
o + t5,9
oi (
```

Cela tourne seulement sur quelques options de configuration utiles, tourne sur la pagination, définit quelques arrêts de tabulation, et désactive certaines des options d'entrée standard. Pour beaucoup de gens, ce fichier d'initialisation simple est tout ce qui est nécessaire.

Un autre ensemble standard de commandes vu dans de nombreux tampons init est

```
zd (t) zd (d) zd (u)
```

Ceux-ci sont utilisés par des personnes qui ne veulent pas les tampons standard fournis par FRED contenant le temps, la date et l'userid de l'utilisateur actuel. Le ZD (Zap Supprimer) efface simplement ce qui est dans le tampon spécifié, de sorte que "zd (u)" efface le tampon "u".

Bien sûr, il est possible de créer un tampon init qui fait beaucoup plus que les choses simples décrites ci-dessus. Beaucoup de gens créent par init tampons qui simulent en fait les actions du FRED bootstrap loader standard. Ci-dessous, nous examinerons un tel programme tampon en détail. Ce programme peut être trouvée dans «fred / sample / .init2". Notez que nous avons élargi, inclus des commentaires, et a fait un ou deux changements mineurs pour rendre le tampon plus lisible. Il y a aussi plusieurs petites différences entre le comportement de ce tampon et le comportement du chargeur de démarrage réelle.

```
o + p + o s {|. (^ $ * [ "options utiles
b (0) un "\ R
\ F "append ligne de commande b (0)
k (.) "ligne de commande de copie à b (.)
s / * / \ C
/ "Briser la ligne de commande
g / ^ $ / d "se débarrasser des lignes nulles
ld "se débarrasser de" fred "
* T ./ "reste quelque chose?
j (b) t "s'il y a args, aller à @ (b)
                "Sinon supprimer la malbouffe et
                "Préparez-vous pour l'édition normale
oq u1 zd (u) zd (d) zd (t) zd (.) zd (f) zd (0) je / Fred /
@ (B) "vous obtenez ici s'il y avait args
t / ^ r $ / "est-ce une commande de lecture?
j (r) t "si oui, aller à @ (r)
m (g) "sinon, déplacez le nom du fichier à b (g)
                "Essayer de lire le fichier en b (.)
b (.) $ r \ B (g), \ B (u) / fred / \ B (g), fred / \ B (g)
                "Erreur si vous ne l'avez pas trouvé
                "Quoi que ce soit maintenant
jt je ;? n'a pas pu accéder \ S (g);
                "Vous obtenez ici si lu ouvrées
zd (g) "se débarrasser du nom de fichier
b (0) oq
u1 zd (f) \ C \ B (.) je //
```

```

                "Supprimer tampon init, exécuter
                "Trucs dans b (.), Et la sortie
@ (R) "que vous obtenez ici si la commande était" r "
ld "supprimer" r "en b (0)
t ./ "il n'y a rien à lire?
                "Sinon, erreur
jt je /? lire quoi?/
lm (g) "sinon, déplacer le nom de fichier à b (g)
b (f) $ -2za (0) "append args b (f)
b (0) * d "effacer un tampon zéro
rx \ B (g) «lire dans le fichier
                "En cas d'erreur, dire pourquoi et la sortie
jt y je //
$ - && T / ^ \ C $ | [^] * \ C (ld [aqx] | fin | octal) /
                "Est-ce un programme GMAP?
                "Le cas échéant, définir des tabulations GMAP
                "Et dire que vous l'avez fait
jf o + t8,16,32,72 jp / o + t8,16,32,72 /
                "Dites vous imprimez la ligne 1
jm / 1p /
1p "et l'imprimer
oq "supprimer la malbouffe et la sortie
b (0) u1 zd (u) zd (d) zd (t) zd (g) zd (.) zd (f) je //

```

Ce tampon fait beaucoup de choses et couvre un certain nombre de situations. Nous allons décrire les actions de la mémoire tampon en termes très généraux, et le laisser au lecteur de comprendre les détails des commentaires donnés ci-dessus.

Tout d'abord le tampon définit certaines options. Voici où vous pouvez spécifier d'autres options si vous le souhaitez. Il ajoute ensuite la ligne de commande entière à tampon zéro et copie la ligne de commande pour "b (.)" Avec un "devant lui afin qu'il sera traité comme un commentaire si" b (.) "Est exécutée. La commande ligne "b (0)" est divisé en changeant chaque chaîne d'ébauches en un retour chariot. ligne 1 contient le mot "fred" à partir de la ligne de commande, ce qui est supprimé ainsi que toutes les lignes nulles. Si le tampon zéro est maintenant vide, la ligne de commande ne se composait que du mot "fred"; par conséquent, ceci est juste une session d'édition simple et tampons inutiles peuvent tous être éliminés le "je / Fred /" commande imprime le familier "Fred" accusé de réception.

S'il y avait plus à la ligne de commande que juste "fred", le tampon voit si la chaîne suivante était un seul "r". Sinon, la chaîne suivante est considérée comme un nom de fichier. Le nom du fichier est copié dans "b (g)" et le tampon init fait plusieurs tentatives pour lire le fichier. Si les tentatives échouent, il imprime un message d'erreur; sinon, il lit le fichier en "b (.)", puis exécute le tampon avec un "\ B (.)" construire dans une boucle U1. La boucle U1 est utilisée pour la même raison que nous avons utilisé la U1 dans l'exemple précédent.

Si la ligne de commande a la forme

```
fred r filename
```

le tampon va trouver son chemin vers l'étiquette "@ (r)". Il vérifie si un "nom de fichier" vraiment été spécifié, et si oui, tente de lire le fichier. Si le fichier d'entrée ressemble GMAP code source, les onglets GMAP standards sont définis pour le fichier. Le tampon affiche la première ligne du fichier, nettoie ordures inutiles, et prépare pour l'édition normale.

Il existe plusieurs façons de modifier ce tampon init pour vos propres besoins. Une des façons les plus simples consiste à modifier les options en haut du fichier les options que vous souhaitez utiliser. Un autre simple changement est d'avoir le tampon imprimer les quelques dernières lignes de votre fichier plutôt que de la première ligne.

Nous pourrions dire que vous pouvez utiliser la "lecture rapide" ligne de commande

```
fred r filename
```

même si vous ne disposez pas de cette fonctionnalité intégrée dans votre tampon init. La raison pour cela est qu'il ya un tampon stocké sous "fred / r" qui fera le lecture rapide pour vous. Ce tampon de lecture est très

similaire à la mémoire tampon init nous venons de parler. Cependant, vous n'êtes pas obligé de le comprendre pour pouvoir l'utiliser. Bien entendu, de même pour le tampon ci-dessus; à condition que vous définissez les bonnes options dans la première ligne, vous pouvez utiliser le tampon init ci-dessus avec un degré élevé de sécurité, même si vous ne comprenez pas toutes les lignes de celui-ci.

Exemple 5: Un tampon pour générer Tampons

Dans cet exemple, nous donnons des moyens dans lequel un tampon peut être stocké dans un fichier et invoqué pour mettre en place un certain nombre d'autres tampons pour vous d'utiliser au cours d'une session de travail.

Une façon simple et directe pour ce faire est de garder vos différents tampons dans des fichiers distincts, un tampon vers un fichier. Par exemple, votre tampon pour créer des tampons peuvent contenir des déclarations comme

```
b (nom1) r / path1
b (nom2) r / path2
```

Si vous mettez ce tampon dans un fichier appelé `"/ start"` (ou `"/ fred / start"`) et invoquer FRED par la commande

```
fred / start
```

fichier `"/ start"` sera lu dans le tampon `"b (.)"` et exécuté.

Sinon, vous pouvez mettre votre tampon pour faire des tampons dans votre fichier init `"de /fred/.init"` et il sera exécuté à chaque fois que vous entrez FRED.

Ces deux approches fonctionnent bien. Cependant, vous pourriez souhaiter garder plusieurs de vos tampons dans un seul fichier. Si oui, vous pourriez avoir un quelque chose de fichier comme ceci:

```
o + s {}
oi (
b. / ^> B / "
uf .k ~ b ~ t / ^> b / jf / s * // s;.. ^> b;., / ^> b / -1m; b. \ C \ B ~
b0 zd ~ u1 zd. je / Prêt: /
> B (nom1) commentaire 1
.
.
.
> B (nom2) commentaire 2
.
.
.
> b
"Fin de fichier
```

La troisième ligne indique FRED que vous souhaitez travailler sur `"b (.)"`. Rappelez-vous cependant que le bootstrapper sera charger `.bf` ce fichier dans le tampon `"."`, Il agira sur elle-même. Ce tampon se coupe en plusieurs tampons. Quand vous faites ce genre d'auto-chirurgie, vous devez faire attention à respecter les règles suivantes:

1. Ne pas modifier la ligne en cours d'exécution.
2. Ne pas ajouter du texte après la dernière ligne.
3. Si la mémoire tampon qui est en cours d'exécution a été appelé à partir d'un autre tampon (par exemple par un `"\ B"` dans le tampon d'appel), ne modifie pas la ligne courante dans le tampon d'appel ou de la ligne après la ligne courante, et ne joignent pas le texte à la fin du tampon d'appel. Le même principe est applicable lorsqu'un tampon appelle un tampon qui appelle un autre tampon et ainsi de suite.
4. Vous ne devez pas modifier la ligne en cours dans l'un des tampons d'appel ou ajouter du texte au bas de ces tampons.

La troisième ligne définit également la ligne en cours d'être la première ligne commençant par "> b". La commande UF crée un tampon caché contenant ces déclarations:

```
.k ~ b ~ t / ^> b / jf / s * // s;.. ^> b;., / ^> b / -1m; b. \ B ~
```

La ligne actuelle est déplacée dans le tampon "~" et le tampon "~" devient le tampon courant. S'il n'y a pas de caractère après la "> b", la commande T définit le registre de condition à FALSE et un saut est effectué à la fin de la jusqu'à ce tampon; le registre de condition sera toujours FALSE, et donc la boucle UF prendra fin. S'il y a quelque chose après le "> b" (représentant un nom de tampon), les substitutions de supprimer les commentaires et remplacer la chaîne "> b" par la chaîne "., / ^> B / -1m". Tampon "~" contiendra alors la ligne

```
., / ^> B / -1m (buffname)
```

Ceci est exécuté sur le tampon "." faisant partie des lignes de tampon "." être déplacé vers le tampon "buffname". Ce processus se poursuit jusqu'à ce que la ligne avec exactement "> b" est trouvé par la commande T. Depuis cette ligne ne correspond pas à l'expression «^> b." (Il n'y a pas de caractère après le "b"), le registre de condition est définie sur FALSE et la boucle UF cesse d'exécution. Comme vous pouvez le voir, de nombreux tampons peuvent être créés de cette façon.

La ligne suivante passe à tampon zéro et supprime tampon "~". Il supprime ensuite tampon "." lors de l'exécution de la mémoire tampon cachée créée par l'instruction U1. Ceci est un peu comme être sur une branche et en réduisant l'arbre, ce qui peut sembler comme une chose dangereuse à faire. Il n'y a pas de catastrophe parce que la déclaration JE arrête l'exécution de tous les tampons et imprime le message "Ready".

Ce tampon a été conçu pour fonctionner sur le tampon "."; vous pouvez le mettre dans un fichier "start" et de l'utiliser de la manière décrite dans l'exemple précédent.

Si vous souhaitez avoir le même effet, mais préfèrent travailler sur le fichier "/fred/.init" vous pouvez remplacer les premières lignes par celles-ci:

```
o {la oi de + (  
bf / ^> b / "  
uf .m ~ b ~ t / ^> b / jf / s * // s;.. ^> b;., / ^> b / -1m; bf \ C \ B ~  
b0 zd ~ jm / Prêt: / u1 jo
```

Maintenant, l'auto-chirurgie est effectuée sur le tampon "f". Notez que, dans la troisième ligne il n'y avait qu'une seule référence au tampon "." et cela a été changé en "f". Notez également que les "zdf" qui auraient pu être dans la quatrième ligne ne sont pas nécessaires parce que cela se fait par le bootstrap.

Exemple 6: Enracinés Trees

Voici un tampon qui détermine si deux arbres enracinés sont isomorphes. Nous utilisons la notation crochets pour représenter les arbres. Pour les définitions de "arbre", "arbre enraciné", et "isomorphes" consulter un bon livre sur la théorie des graphes comme «Théorie des graphes avec des applications" par JABondy et USRMurty (publié en 1976 par Macmillan de Londres et Elsvier). Voici deux arbres à racines non-isomorphes et leurs représentations du support.

```
ABABC  
 \ / \ / /  
  \ / \ / /  
   . C. .  
    \ / \ /  
     \ / \ /  
      . .
```

```
(( (A) (B) ) (C) ) (( (A) (B) ) ((C)))
```

Voici tampon "arbres".

```

"=== / Fred / arbres === ljdickey 28/08/1980 17:20
"contenu du tampon d'essai" 0 "
«deux arbres enracinés isomorphes
"Les supports peuvent être de ces types:
"(), <>, {}, Ou [].
o [la Oi + (
b0 * k1 b1 * s / [([] / </ * s / []) \ C]] /> /
* S / [^ <>] // * s / <> / <:> / * s ./& \ C
/ * S / \ C
//
uf $ - / ^ <$ / , / ^> $ / m2 b2 1d $ d * zs \ C
      b1 i <\ C \ L2> \ C \ F 1t / ^ <$ /
$ N (tt) un (tt) = 2 jt je / ** Non 2 trees./
1m2 t / ^ \ L2 $ / jt je / Les arbres ne sont pas isomorphic./
je / Les deux arbres sont isomorphic./

```

Si vous avez lu jusqu'ici, il est clair que vous êtes sur votre chemin pour devenir un duffer tampon et aurez plaisir à travailler sur une explication de chacune des étapes pour vous-même. Vous verrez qu'une copie des données reste dans un tampon "0", que tout le travail est effectué dans des tampons "1" et "2", que tous les supports sont remplacés par "<" et ">", et que tous les caractères sont placés sur une ligne par lui-même sans retour chariot. Vous devrez payer une attention particulière à l'utilisation du symbole "\$" qui correspond à la valeur nulle (imaginaire) de caractère à la fin de chaque ligne, même quand il n'y a pas un retour chariot à la fin de la ligne. Le cœur de la méthode consiste à utiliser le genre "* zs" pour mettre la représentation dans une forme canonique.

Remerciements:

Les auteurs tiennent à remercier Michael A. Afheldt, William CW Ince, et Peter J. Fraser pour leur aimable assistance en lisant les versions préliminaires de ce livret et suggérant des changements qui ont amélioré lui.